| | |
|---|---|
| **Universidad Galileo** | **Guatemala 18 de Febrero del 2026** |
| **Facultad: FISICC** | **Alumno: Cruz Israel Ambrocio Tepax** |
| **Curso: CC VII** | **Carnet: 20005588** |
| **Sección: AN** | **Hora de Laboratorio: 6:00 PM** |
| **Auxiliar:** | **Día de Laboratorio: viernes** |

# Laboratorio # 1 Process Creation and Management

## Objetivos:

El objetivo de este laboratorio es entender cómo se crean y manejan los procesos en un sistema operativo, ya sea Linux o Windows. Se trabaja con llamadas al sistema como fork() y exec() en Linux o CreateProcess() en Windows para crear procesos hijos, ver cómo se relacionan con el proceso padre, y entender en qué orden se ejecutan. También se explora cómo sincronizar procesos (es decir, hacer que el padre espere a que el hijo termine) y cómo comunicar procesos entre sí usando mecanismos como pipes y memoria compartida.

## Resumen:

En este lab se desarrollan varios programas pequeños que van incrementando en complejidad. Se empieza creando un proceso hijo simple que imprime su PID junto con el del padre, luego se modifica para que el padre espere a que el hijo termine antes de continuar. Después se implementa comunicación entre procesos usando pipes, donde el padre envía un mensaje y el hijo lo recibe. También se trabaja con la creación de múltiples procesos hijos usando un loop, y opcionalmente se puede implementar memoria compartida para intercambiar datos entre procesos.

## Teoría:

Un proceso es básicamente un programa en ejecución. Cuando un sistema operativo corre un programa, le asigna un espacio de memoria, un identificador único (PID) y los recursos que necesita para funcionar. En Linux, los procesos nuevos se crean con la llamada al sistema fork(), que genera una copia casi exacta del proceso que la invoca. El proceso original se convierte en el padre y la copia en el hijo. Cada uno sigue ejecutándose de forma independiente, pero el hijo hereda cosas como las variables y los descriptores de archivo del padre. Esto permite que un solo programa se divida en dos flujos de ejecución distintos.

La sincronización entre procesos es importante porque cuando un padre crea un hijo, ambos corren de manera concurrente y no hay garantía de cuál termina primero. Si el padre necesita el resultado del hijo o simplemente necesita esperar a que termine, se usa waitpid() en Linux o WaitForSingleObject() en Windows. Sin esta sincronización, el padre podría terminar antes que el hijo, dejando procesos huérfanos, o el hijo podría terminar sin que nadie recoja su estado de salida, creando lo que se conoce como procesos zombie.

La comunicación entre procesos (IPC) permite que dos o más procesos intercambien datos. Uno de los mecanismos más básicos es el pipe, que funciona como un canal unidireccional: un proceso escribe en un extremo y otro lee del otro. Es útil para pasar mensajes simples entre padre e hijo. Para escenarios más avanzados existe la memoria compartida, donde ambos procesos acceden a una misma región de memoria RAM, lo que permite intercambiar datos de forma mucho más rápida que con pipes, aunque requiere más cuidado para evitar condiciones de carrera.

-

## Resultados:

Creating a New Process

```
cruz@Cruzs-MacBook-Pro Code (main) % ./build_and_run.sh task1_fork
Compiling task1_fork.c...
Built bin/task1_fork
Running bin/task1_fork...
---
Parent Process: PID=17144
Child Process: PID=17145, Parent PID=17144
cruz@Cruzs-MacBook-Pro Code (main) %
```

Synchronizing Parent and Child Process

```
cruz@Cruzs-MacBook-Pro Code (main) % ./build_and_run.sh task2_sync
Compiling task2_sync.c...
Built bin/task2_sync
Running bin/task2_sync...
---
Child Process: PID=17467, Parent PID=17466
Parent Process: Child has finished execution.
cruz@Cruzs-MacBook-Pro Code (main) %
```

Inter-Process Communication Using Pipes

```
cruz@Cruzs-MacBook-Pro Code (main) % ./build_and_run.sh task3_pipe
Compiling task3_pipe.c...
Built bin/task3_pipe
Running bin/task3_pipe...
---
Parent Process: Writing "Hello from Parent"
Child Process: Received "Hello from Parent"
cruz@Cruzs-MacBook-Pro Code (main) %
```

Creating Multiple Child Processes

```
cruz@Cruzs-MacBook-Pro Code (main) % ./build_and_run.sh task4_multi
Compiling task4_multi.c...
Built bin/task4_multi
Running bin/task4_multi...
---
Parent Process: PID=17844
Child 1: PID=17845, Parent PID=17844
Child 2: PID=17846, Parent PID=17844
Child 3: PID=17847, Parent PID=17844
cruz@Cruzs-MacBook-Pro Code (main) %
```

Shared Memory (Advanced Task)

```
cruz@Cruzs-MacBook-Pro Code (main) % ./build_and_run.sh task5_shm
Compiling task5_shm.c...
Built bin/task5_shm
Running bin/task5_shm...
---
Parent Process: Writing "Shared Memory Example"
Child Process: Read "Shared Memory Example"
cruz@Cruzs-MacBook-Pro Code (main) %
```

## Conclusiones:

- En este laboratorio se pudo comprobar que la creación de procesos en Linux mediante fork() es un mecanismo sencillo pero poderoso, ya que permite dividir la ejecución de un programa en múltiples flujos independientes que comparten un origen común. Entender cómo se relacionan los procesos padre e hijo, y cómo sincronizarlos correctamente, es fundamental para evitar problemas como procesos zombie o comportamientos inesperados.
- 
- Además, se logró entender cómo los mecanismos de IPC como pipes y memoria compartida permiten que procesos separados colaboren entre sí. Estos conceptos son la base de cómo funcionan los sistemas operativos modernos, donde múltiples procesos necesitan comunicarse y coordinarse constantemente para que todo funcione de forma estable.

## E-grafía:


- Linux man-pages project. *fork(2) — Linux manual page*. Recuperado de: https://man7.org/linux/man-pages/man2/fork.2.html